

# Lab 2: Data Preprocessing

## Objective:

The objective of this lab is to equip students with practical skills in essential data preprocessing steps for machine learning, including data cleaning, handling outliers, feature scaling, feature engineering, and data splitting. By using the dataset, students will learn to address common issues such as missing values and duplicates, scale features for consistency, create new features, and properly split data into training and testing sets. This hands-on exercise will demonstrate how these preprocessing tasks are crucial for preparing data for effective machine learning modeling.

## Software Requirements:

- Python
- scikit-learn
- pandas
- numpy
- matplotlib

## Dataset:

You may use the New York City Airbnb Dataset which is a popular dataset used for data analysis, machine learning, and various other data science applications. The dataset contains information about Airbnb listings in New York City, specifically for January 2025. It includes various features related to the listings, hosts, reviews, and more, providing insights into the properties available for short-term rentals. There are 429 samples and 75 feature variables.

## Lab Instructions

### 1. Import necessary libraries, for example:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

### 2. Load the dataset:

For datasets available in Scikit-learn like Boston Housing Dataset, you may load by package, for example:

```
from sklearn.datasets import load_boston
data = load_boston()
```

For external datasets, you may use pandas to load it, for example:

```
data = pd.read_csv('path_to_dataset.csv')
```

### 3. Follow the exercises below to master data preprocessing skills.

#### 1) Data Loading and Exploration: (5 pts)

- Use Pandas to load the dataset.
- Display the first 5 rows of the dataset.
- Describe the dataset's statistical properties (mean, median, standard deviation).

first 5 rows & statistical properties

#### 2) Missing Values Diagnosis: (5 pts)

- Check for any missing values and print their column names and the corresponding number of missing values. Then present the data missing rate using a bar chart.

#### 3) Data Cleaning: (30 pts)

- Handle missing values if there are any(e.g., drop rows, fill with mean/median...):
  - Fill in the missing values of the 'price' with the mean value.
  - Fill in missing values of 'description' with 'Not Provided'.
  - Fill in the missing value of 'host\_response\_time' with mode value.
  - Fill the missing values in the column 'host\_is\_superhost' and convert it into one-hot encoding.

Print the number of remaining missing values for each column.

- Remove duplicates and handle any inconsistencies in the data. Print the number of duplicate data.
- Convert 'neighbourhood' to one-hot encoding and print 'neighbourhood'.

#### 4) Outlier Detection and Treatment: (15 pts)

- Visualize the distribution of 'price' using histplot.
- Use the Interquartile Range (IQR) method to remove or cap extreme outliers.
- Visualize the 'price' distribution after IQR processing using hisplot and briefly analyze the differences between the two distributions.

#### [Supplement to IQR]

We calculate the first and third quartiles (Q1 and Q3), and then use the difference between Q3 and Q1 (IQR) to determine if data points are outliers. Typically, any data points below  $Q1 - 1.5 * IQR$  or above  $Q3 + 1.5 * IQR$  are considered outliers. The implementation of IQR is as follows:

```

# Calculate IQR
Q1 = df['price'].quantile(0.25)
Q3 = df['price'].quantile(0.75)
IQR = Q3 - Q1

# Calculate lower and upper limits
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

# Filter out outliers
df = df[(df['price'] >= lower_limit) & (df['price'] <= upper_limit)]

```

#### 5) Feature Scaling: (10 pts)

- Apply Min-Max Scaling for features like 'price' to bring them between 0 and 1. And store the scaled data in the newly created 'price\_scaled' column. Print the maximum and minimum values of 'price\_scaled'.
- Apply Standardization to other features like 'number\_of\_reviews'. And store the processed data in the newly created column 'number\_of\_reviews\_scaled'. Print the mean and standard deviation of 'number\_of\_reviews\_scaled'.

#### 6) Feature Engineering (10 pts)

- Create a new column 'Price\_per\_Bedroom' and fill in the relevant data.
- Create a new column 'description\_word\_count' and fill in the relevant data. Print the first 5 rows of data for the columns: price, bedrooms, Price\_per\_Bedroom, description, description\_word\_count

#### 7) Data Splitting (5 pts)

- Split the dataset into features (x) and target variable (y). In this workshop, x is all columns without 'price', while y is 'price'.
- Split the data into training (80%) and test (20%) sets. Hint: may use 'from sklearn.model\_selection import train\_test\_split'.

#### 8) Building a Pipeline with Iris Dataset (20 points)

- Import the Iris dataset built into scikit-learn, create and apply a pipeline. The specific steps are: ① Importing the iris dataset and dividing it into features and targets. ② Divide the training set and testing set in a 7:3 ratio. ③ Create a Pipeline, fill in missing values with the mean, standardize them, and use a logistic regression model with "max\_iter=200" for classification. ④ Train model. ⑤ Predict and evaluate model. ⑥ Output the model accuracy (to 2 decimal places) and visualize the model performance using a confusion matrix.

**Submission Requirement:**

- 1) Complete section 3 in a jupyter notebook file, name your file as “lab2\_student number.ipynb” and submit it on iSpace.
- 2) Please provide necessary outputs to support your assignment. (e.g. printing necessary information or visualizing through graphs and tables)
- 3) Late submission and missing outputs will result in point deductions even if the code is correct.

**Deadline: 21:00, Mar. 3, 2026, Plagiarism (cheating) = F**